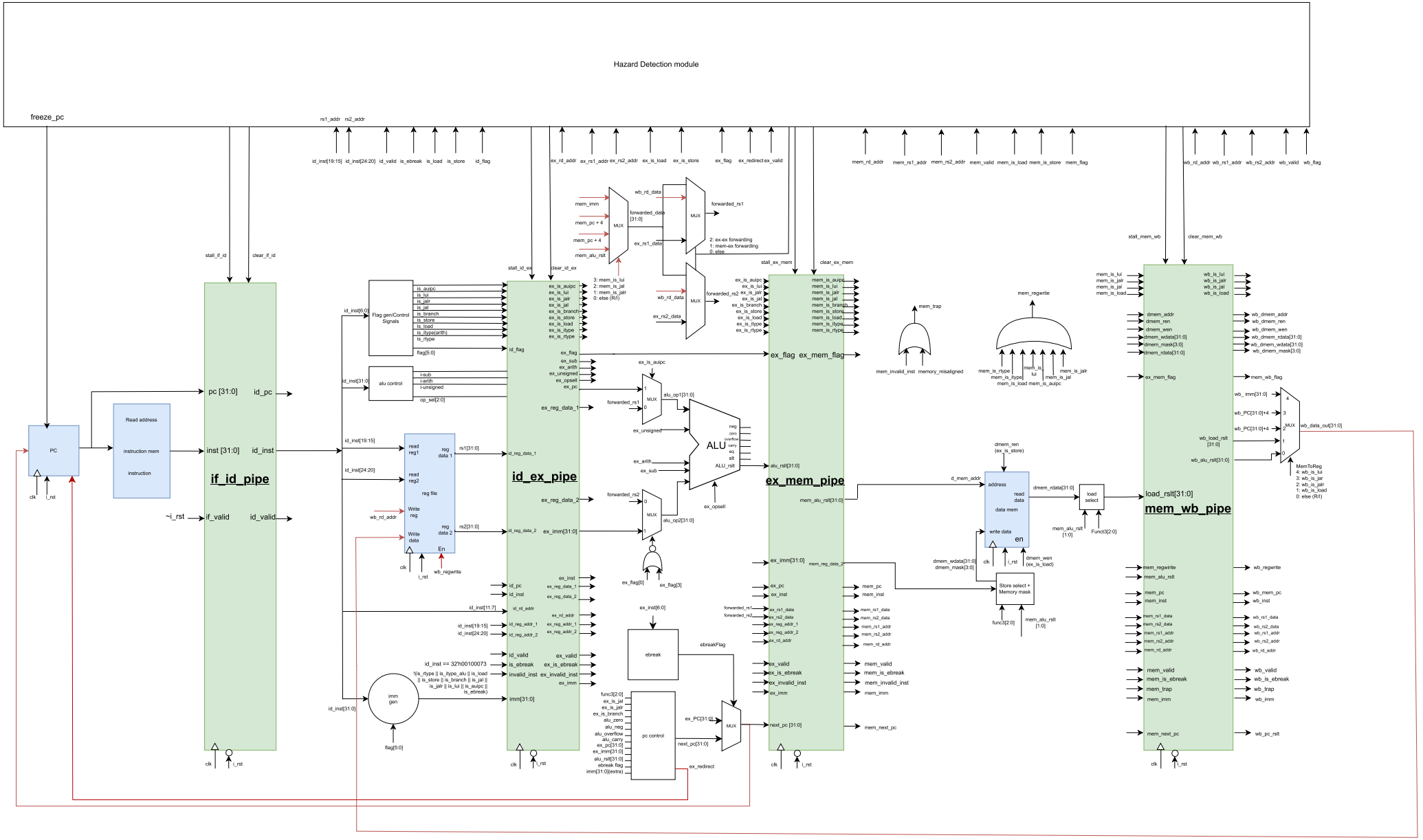

Hazard Detection module
freeze_pc
stall_if_id
clear_if_id
pc [31:0]
id_pc
inst [31:0]
id_inst
if_id_pipe
~i_rst
if_valid
id_valid
PC
Read address
instruction mem
instruction
clk
i_rst
id_inst[19:15]
id_inst[24:20]
id_inst[11:7]
id_inst[31:0]
id_inst[6:0]
Flag gen/Control Signals
alu control
op_sel[2:0]
flag[5:0]
read reg1
read reg2
reg file
Write reg
Write data
reg data 1
reg data 2
rs1[31:0]
rs2[31:0]
wb_rd_addr
wb_regwrite
En
imm gen
id_imm[31:0]
stall_id_ex
clear_id_ex
id_ex_pipe
id_flag
id_reg_data_1
id_reg_data_2
id_rd_addr
id_reg_addr_1
id_reg_addr_2
id_valid
is_ebreak
invalid_inst
id_imm
ex_flag
ex_reg_data_1
ex_reg_data_2
ex_rd_addr
ex_reg_addr_1
ex_reg_addr_2
ex_valid
ex_is_ebreak
ex_invalid_inst
ex_imm
ex_inst
ex_pc
ex_rs1_addr
ex_rs2_addr
ex_rd_addr
ex_is_load
ex_is_store
ex_flag
ex_redirect
ex_valid
forwarded_rs1
forwarded_rs2
forwarded_data [31:0]
2: ex-ex forwarding
1: mem-ex forwarding
0: else
3: mem_is_lui
2: mem_is_jal
1: mem_is_jalr
0: else (R/I)
alu_op1[31:0]
alu_op2[31:0]
ALU
ALU_rslt
ebreak
ebreakFlag
pc control
ex_PC[31:0]
next_pc[31:0]
ex_redirect
stall_ex_mem
clear_ex_mem
ex_mem_pipe
ex_flag ex_mem_flag
alu_rslt[31:0]
mem_rd_addr
mem_rs1_addr
mem_rs2_addr
mem_valid
mem_is_load
mem_is_store
mem_flag
mem_trap
mem_invalid_inst
memory_misaligned
mem_regwrite
mem_is_rtype
mem_is_itype
mem_is_lui
mem_is_jalr
mem_is_load
mem_is_auipc
dmem_ren (ex_is_store)
d_mem_addr
address
read data
data mem
write data
en
dmem_wen (ex_is_load)
dmem_wdata[31:0]
dmem_mask[3:0]
dmem_rdata[31:0]
load select
Funct3[2:0]
mem_alu_rslt [1:0]
Store select + Memory mask
func3[2:0]
stall_mem_wb
clear_mem_wb
mem_wb_pipe
load_rslt[31:0]
wb_load_rslt [31:0]
wb_alu_rslt[31:0]
wb_data_out[31:0]
MemToReg
4: wb_is_lui
3: wb_is_jar
2: wb_is_jalr
1: wb_is_load
0: else (R/I)
wb_dmem_addr
wb_dmem_ren
wb_dmem_wen
wb_dmem_rdata[31:0]
wb_dmem_wdata[31:0]
wb_dmem_mask[3:0]
wb_regwrite
wb_mem_pc
wb_inst
wb_rs1_data
wb_rs2_data
wb_rs1_addr
wb_rs2_addr
wb_rd_addr
wb_valid
wb_is_ebreak
wb_trap
wb_imm
wb_pc_rslt